

Pipeline Walkthrough

Purpose

Operational step-through of the NARWC-DB validation pipeline for curators and developers. Covers loading a survey CSV, running validation, inspecting warnings, writing overrides (per-row and per-survey), and confirming the override took effect. This is the canonical path for a personal pipeline shakedown before onboarding team members. See docs/warning_overrides.md for conceptual background on the override system.

Important: `narwc.validation.SurveyValidator()` called with no arguments uses `load_config()`'s defaults, where `validation.overrides.csv_path` is empty — no override file is read at all in that case. Every example below constructs the validator with an explicit override path instead, so the override workflow actually takes effect. In a real batch run this path comes from the active batch config (e.g. `config/batches/migration.m` sets it to `config/overrides/migration_overrides.csv`) rather than being passed by hand.

Prerequisites

- MATLAB R2020b or later
 - Database Toolbox installed (not required for validation-only steps; required for DB write)
 - data/tables/ populated with lookup CSVs (shipped in the repo)
 - startup.m has been run in the current MATLAB session
-

Quick Start

```
startup();
override_path = fullfile('config', 'overrides', 'walkthrough_overrides.csv');
parser = narwc.io.parsers.StandardFormat();
[data, ~] = parser.parse('tests/fixtures/sample_data/oT06129.csv');
validator = narwc.validation.SurveyValidator(struct('override_file', override_path));
[is_valid, results] = validator.validate(data);
results.summary
```

Expect: `is_valid` is true or false; `results.summary` shows error/warning counts. (`walkthrough_overrides.csv` doesn't need to exist yet — `loadOverrides` returns no overrides, not an error, when the file is missing.)

Step-by-Step Walkthrough

Step 1 — Setup

```
startup();  
override_path = fullfile('config', 'overrides', 'walkthrough_overrides.csv');
```

What to observe: No error output. `load_config` and `logging.Logger` should be on the path. If you see Undefined function 'load_config', re-run startup from the repo root.

Step 2 — Load a Survey CSV

```
file_path = 'tests/fixtures/sample_data/oT06129.csv';  
parser = narwc.io.parsers.StandardFormat();  
[data, metadata] = parser.parse(file_path);  
fprintf('Loaded %d rows, %d columns\n', height(data), width(data));  
disp(metadata)
```

What to observe: `data` is a MATLAB table with 55 named columns in database order. `metadata.format` is 'Standard NARWC Format'. `metadata.row_count` matches the file's row count.

Step 3 — Validate the Loaded Survey

```
validator = narwc.validation.SurveyValidator(struct('override_file', override_path));  
[is_valid, results] = validator.validate(data);  
disp(results.summary)
```

What to observe: `results.summary` prints error/warning counts. `is_valid` is false if any unacknowledged warnings or errors exist (default config blocks on warnings). `results.error_details` is a cell array of formatted strings for quick reading.

Print all detail lines:

```
for i = 1:length(results.error_details)  
    disp(results.error_details{i})  
end
```

Step 4 — Inspect Individual Warnings

`results.warnings` is a struct array. Each element has: `field`, `row`, `eventno`, `rule_id`, `message`, `severity`.

```
% How many warnings?
n = length(results.warnings);
fprintf('%d warning(s)\n', n);
```

```
% Inspect the first warning
w = results.warnings(1);
fprintf('Field:   %s\n', w.field);
fprintf('EVENTNO: %d\n', w.eventno);
fprintf('Rule:    %s\n', w.rule_id);
fprintf('Message: %s\n', w.message);
```

What to observe: `w.eventno` is the integer EVENTNO you need to write a per-row override. `w.rule_id` is the stable rule identifier (e.g., `behavioral_rules.calf_behavior_no_calf`). Both values go directly into the override CSV.

To list all warnings in a readable format:

```
for i = 1:length(results.warnings)
    w = results.warnings(i);
    fprintf('%d] EVENTNO=%-6s field=%-12s rule=%s\n', ...
        i, num2str(w.eventno), w.field, w.rule_id);
end
```

Step 5 — Write a Per-Row Override and Re-Validate

A per-row override suppresses a specific warning for a specific EVENTNO in a specific survey. Fill in the values from Step 4.

Open `override_path` (`config/overrides/walkthrough_overrides.csv`) in a text editor and append one line (create the file with just a header row first if it doesn't exist):

```
fileid,eventno,field,rule_id,acknowledged_by,acknowledged_date,reason
oT06129,<EVENTNO>,<field>,<rule_id>,rss,2026-06-26,walkthrough test override
```

Replace `<EVENTNO>`, `<field>`, and `<rule_id>` with the values from Step 4.

Then re-validate (a new validator instance is needed to reload the overrides file):

```
validator2 = narwc.validation.SurveyValidator(struct('override_file', override_path)
[is_valid2, results2] = validator2.validate(data);
fprintf('Warnings after per-row override: %d new, %d acknowledged\n', ...
    results2.summary.warnings_new, results2.summary.warnings_acknowledged_per_row);
```

Note, you need to make a new `SurveyValidator` instance or it will not reload the overrides file.

What to observe: results2.summary.warnings_acknowledged_per_row increments by 1. The overridden warning moves from results2.warnings to results2.info. If warnings_new drops to 0 and errors is 0, is_valid2 becomes true.

Step 6 — Write a Per-Survey Override and Re-Validate

A per-survey override suppresses all warnings of a given (field, rule_id) combination for a survey, regardless of EVENTNO. Leave the eventno column empty.

Append to override_path:

oT06129,,<field>,<rule_id>,rss,2026-06-26,per-survey walkthrough test

Re-validate:

```
validator3 = narwc.validation.SurveyValidator(struct('override_file', override_path)
[is_valid3, results3] = validator3.validate(data);
fprintf('Per-survey acknowledged: %d\n', results3.summary.warnings_acknowledged_per_
```

What to observe: results3.summary.warnings_acknowledged_per_survey reflects how many warnings were silenced by the per-survey rule. See docs/warning_overrides.md for override precedence.

Step 7 — Run Validation on the High-Volume Fixture

```
file_vol = 'tests/fixtures/sample_data/aT99001_volume.csv';
parser_vol = narwc.io.parsers.StandardFormat();
[data_vol, meta_vol] = parser_vol.parse(file_vol);
fprintf('Volume fixture: %d rows\n', meta_vol.row_count);
```

```
validator_vol = narwc.validation.SurveyValidator(struct('override_file', override_pa
[is_valid_vol, results_vol] = validator_vol.validate(data_vol);
disp(results_vol.summary)
```

Add a line to override_path:

aT99001,,BEHAV,behavioral_rules.calf_behavior_no_calf,rjs,2026-06-26,walkthrough test override

And run again.

What to observe: Validation completes without timeout. results_vol.summary shows the aggregate counts across all rows. This fixture is used to catch performance regressions; a reasonable run should complete in a few seconds.

Step 8 — Run the Smoke Test Driver

`scripts/validate_fixtures.m` runs validation over every survey fixture in `tests/fixtures/sample_data/` and prints a summary table.

```
startup();    % if not already done
validate_fixtures()
```

What to observe: A table is printed with one row per fixture file showing ROWS, ERRORS, WARN, ACK_ROW, ACK_SURV, VALID, and elapsed time. TOTALS row at the bottom. Any fixture in the FAILED state indicates a parser or validator crash — investigate that fixture before proceeding. Correct it by investigating each and adding a line to your override CSV.

Step 9 — Cleanup

Remove the walkthrough override entries added in Steps 5–7. Open `override_path` and delete the lines containing `walkthrough test override` and `per-survey walkthrough test`. Alternatively, reset to header-only:

```
fid = fopen(override_path, 'w');
fprintf(fid, 'fileid,eventno,field,rule_id,acknowledged_by,acknowledged_date,reason\n');
fclose(fid);
```

What to observe: A fresh `narwc.validation.SurveyValidator(struct('override_file', override_path))` will load 0 overrides. The log line `Loaded N override(s)` will not appear.

Common Operations

How do I find the EVENTNO for a warning I want to acknowledge?

```
for i = 1:length(results.warnings)
    w = results.warnings(i);
    fprintf('EVENTNO=%-6s field=%-12s rule=%s\n', num2str(w.eventno), w.field, w.rule);
end
```

How do I check whether my override took effect?

```
validator = narwc.validation.SurveyValidator(struct('override_file', override_path));
[~, results] = validator.validate(data);
fprintf('Per-row acknowledged: %d\n', results.summary.warnings_acknowledged_per_row);
fprintf('Per-survey acknowledged: %d\n', results.summary.warnings_acknowledged_per_survey);
% Acknowledged warnings appear in results.info, not results.warnings
```

How do I see what warnings appeared in the most recent run?

```

for i = 1:length(results.error_details)
    disp(results.error_details{i})
end
% Or inspect the raw struct array:
% disp(results.warnings)

```

How do I remove an override I added by mistake?

Open the override CSV in a text editor and delete the relevant line. Alternatively, view current overrides:

```

t = readtable(override_path, 'CommentStyle', '#', 'Delimiter', ',', ...
    'TextType', 'char', 'VariableNamingRule', 'preserve');
disp(t)

```

Then delete the row by line number in the file.

How do I reset an override CSV to header-only?

```

fid = fopen(override_path, 'w');
fprintf(fid, 'fileid,eventno,field,rule_id,acknowledged_by,acknowledged_date,reason\n');
fclose(fid);

```

Tuning validation thresholds

The validator flags unusually large NUMBER or NUMCALF values per row. Thresholds are per-species or per-taxonomic-group, stored in data/tables/SPECCODE.csv and data/tables/TAXCODE.csv.

To raise the threshold for a specific species:

1. Edit data/tables/SPECCODE.csv
2. Find the row for that species (by Value column, e.g., SAD0 for Atlantic spotted dolphin)
3. Set typical_max_group to your desired threshold integer
4. Save the CSV
5. Run scripts/setup/push_lookup_tables.m to update the database

If a SPECCODE row has NULL for typical_max_group, the validator falls back to the typical_max_group for that species' TAXCODE (taxonomic group).

To change the default for an entire taxonomic group, edit data/tables/TAXCODE.csv the same way. Starting defaults:

TAXCODE	Category	typical_max_group	typical_max_calf
1	Large cetacean	50	10
2	Medium cetacean	200	30

TAXCODE	Category	typical_max_group	typical_max_calf
3	Small cetacean	2000	200
4	Other marine mammal	5000	500
5	Sea turtle	20	5
8	Bird	100000	—
9	Other/unknown	1000	100

For details on the three-level cascade (SPECICODE → TAXCODE → global default), see docs/validation_rules_guide.md.

Troubleshooting

(Populate this section during your personal pipeline shakedown.)

Next Steps

- Read docs/warning_overrides.md for the full override semantics (per-row vs. per-survey, precedence, the acknowledged_by / reason fields).
- Review handoffs/smoke_test_findings.md for known fixture-specific issues and expected warning counts.
- For validation rule details, see docs/validation_rules_guide.md.